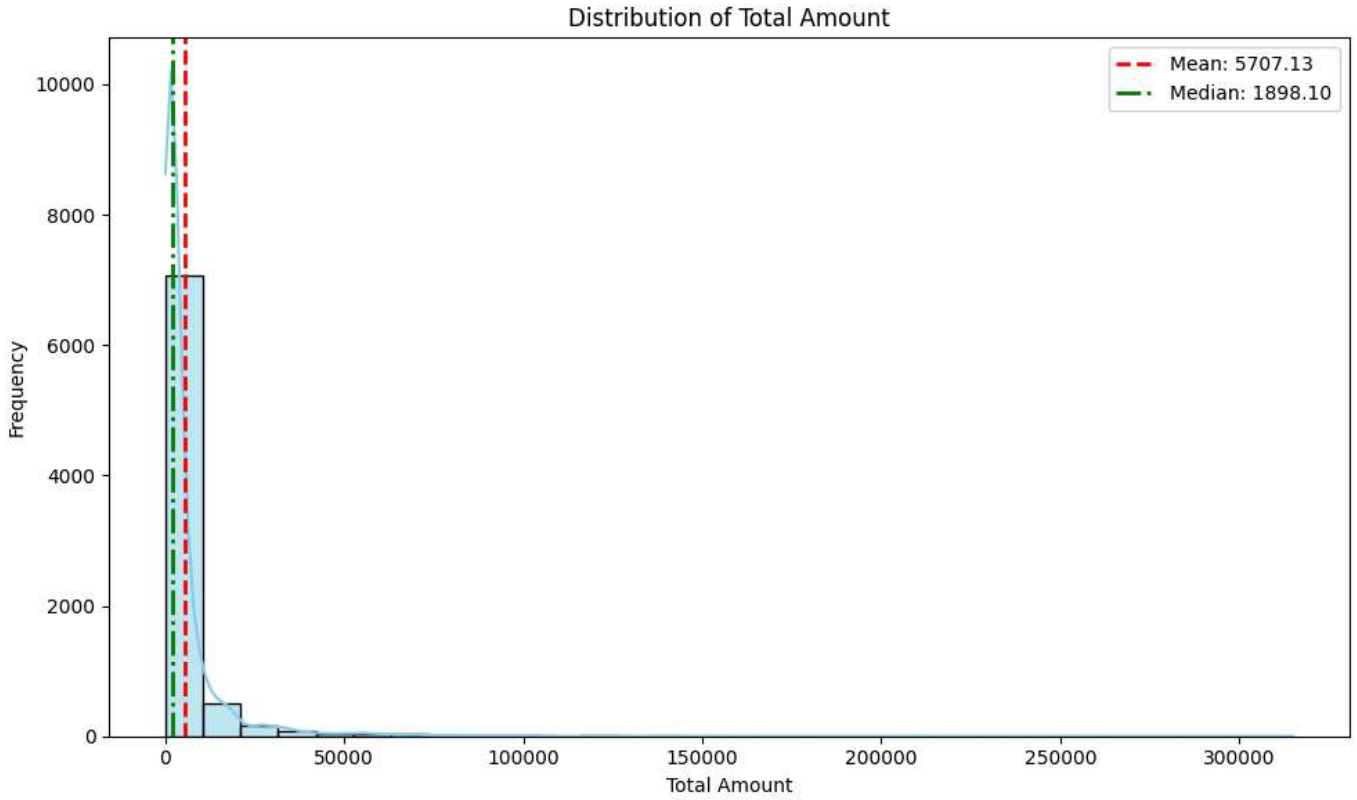
```
#
import pandas as pd
import os
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
folder_path = "/content/"
customers = pd.read_csv(os.path.join(folder_path, "customers.csv"))
marketing = pd.read_csv(os.path.join(folder_path, "marketing.csv"))
orders    = pd.read_csv(os.path.join(folder_path, "orders.csv"))
products  = pd.read_csv(os.path.join(folder_path, "products.csv"))
returns   = pd.read_csv(os.path.join(folder_path, "returns.csv"))
```

```
# Calculate mean, median, mode of total_amount. Plot a histogram and comment on the distribution shape (skewness/kurtosis).
import pandas as pd
import matplotlib.pyplot as plt
import seaborn as sns
mean_val   = orders["total_amount"].mean()
median_val = orders["total_amount"].median()
mode_val   = orders["total_amount"].mode()[0]
print("Mean:", mean_val)
print("Median:", median_val)
print("Mode:", mode_val)
plt.figure(figsize=(10,6))
sns.histplot(orders["total_amount"], bins=30, kde=True, color="skyblue")
plt.axvline(mean_val, color="red", linestyle="--", linewidth=2, label=f"Mean: {mean_val:.2f}")
plt.axvline(median_val, color="green", linestyle="-.", linewidth=2, label=f"Median: {median_val:.2f}")
plt.title("Distribution of Total Amount")
plt.xlabel("Total Amount")
plt.ylabel("Frequency")
plt.legend()
plt.tight_layout()
plt.show()
```

```
Mean: 5707.12928125
Median: 1898.1
Mode: 649.0
```




```
# Calculate variance, standard deviation, IQR, and coefficient of variation (CV) for delivery_days. What does CV tell you about consistency?
import numpy as np
variance_val = orders["delivery_days"].var()
std_val = orders["delivery_days"].std()
Q1 = orders["delivery_days"].quantile(0.25)
Q3 = orders["delivery_days"].quantile(0.75)
IQR = Q3 - Q1
mean_val = orders["delivery_days"].mean()
CV = std_val / mean_val
print("Variance:", variance_val)
print("Standard Deviation:", std_val)
print("IQR:", IQR)
print("Coefficient of Variation (CV):", CV)
```

```
Variance: 11.860512549618111
Standard Deviation: 3.443909486269654
IQR: 6.0
Coefficient of Variation (CV): 0.5266348051267952
```

```
# Detect outliers in total_amount using the IQR method (1.5×IQR rule). How many rows are outliers? What % of total?
Q1 = orders["total_amount"].quantile(0.25)
Q3 = orders["total_amount"].quantile(0.75)
IQR = Q3 - Q1
lower_fence = Q1 - 1.5 * IQR
upper_fence = Q3 + 1.5 * IQR
outliers = orders[
    (orders["total_amount"] < lower_fence) |
    (orders["total_amount"] > upper_fence)
]
outlier_count = outliers.shape[0]
total_count = orders.shape[0]
outlier_pct = (outlier_count / total_count) * 100
print("Outlier count:", outlier_count)
print("Outlier percentage:", outlier_pct)
```

```
Outlier count: 960
Outlier percentage: 12.0
```

```
# Compute z-scores for total_amount. Flag z > 3 as high-value anomalies. Cross-check with IQR results.
mean_val = orders["total_amount"].mean()
std_val  = orders["total_amount"].std()
orders["z_score"] = (orders["total_amount"] - mean_val) / std_val
z_outliers = orders[orders["z_score"] > 3]
z_outlier_count = z_outliers.shape[0]
print("High-value anomalies (z > 3):", z_outlier_count)
Q1 = orders["total_amount"].quantile(0.25)
Q3 = orders["total_amount"].quantile(0.75)
IQR = Q3 - Q1
lower_fence = Q1 - 1.5 * IQR
upper_fence = Q3 + 1.5 * IQR
iqr_outliers = orders[
    (orders["total_amount"] < lower_fence) |
    (orders["total_amount"] > upper_fence)
]
iqr_outlier_count = iqr_outliers.shape[0]
print("IQR outliers:", iqr_outlier_count)
common_outliers = set(z_outliers.index).intersection(set(iqr_outliers.index))
print("Common outliers (both methods):", len(common_outliers))
```

```
High-value anomalies (z > 3): 154
```

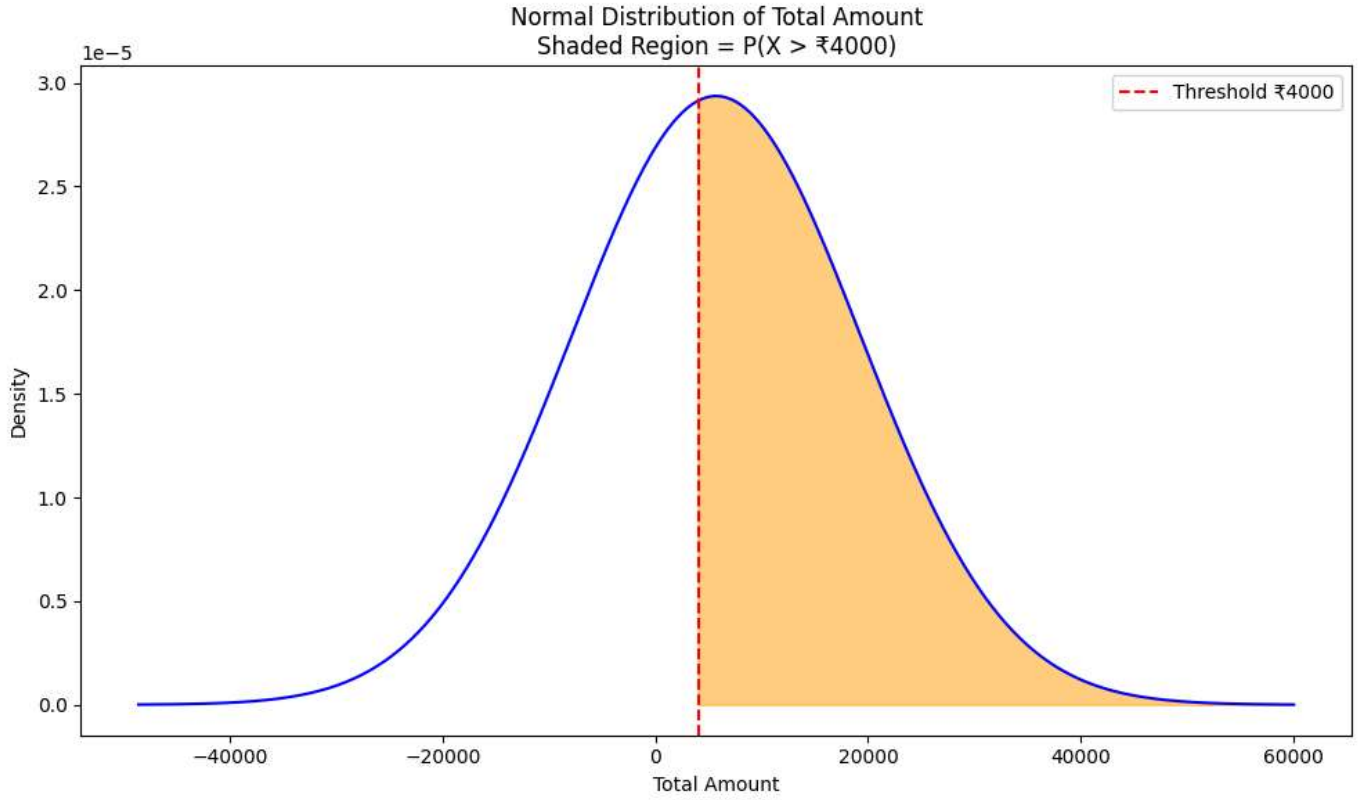
```
# Model order count per customer as a Binomial distribution. What's the probability a randomly chosen customer placed ≥ 3 orders?
import numpy as np
from scipy.stats import binom
n = 10
p = orders["order_id"].count() / (n * orders["customer_id"].nunique())
# P(X >= 3) = 1 - P(X <= 2)
prob_ge3 = 1 - binom.cdf(2, n, p)
print("Probability customer placed ≥ 3 orders:", prob_ge3)
```

```
Probability customer placed ≥ 3 orders: 0.8442450064127331
```

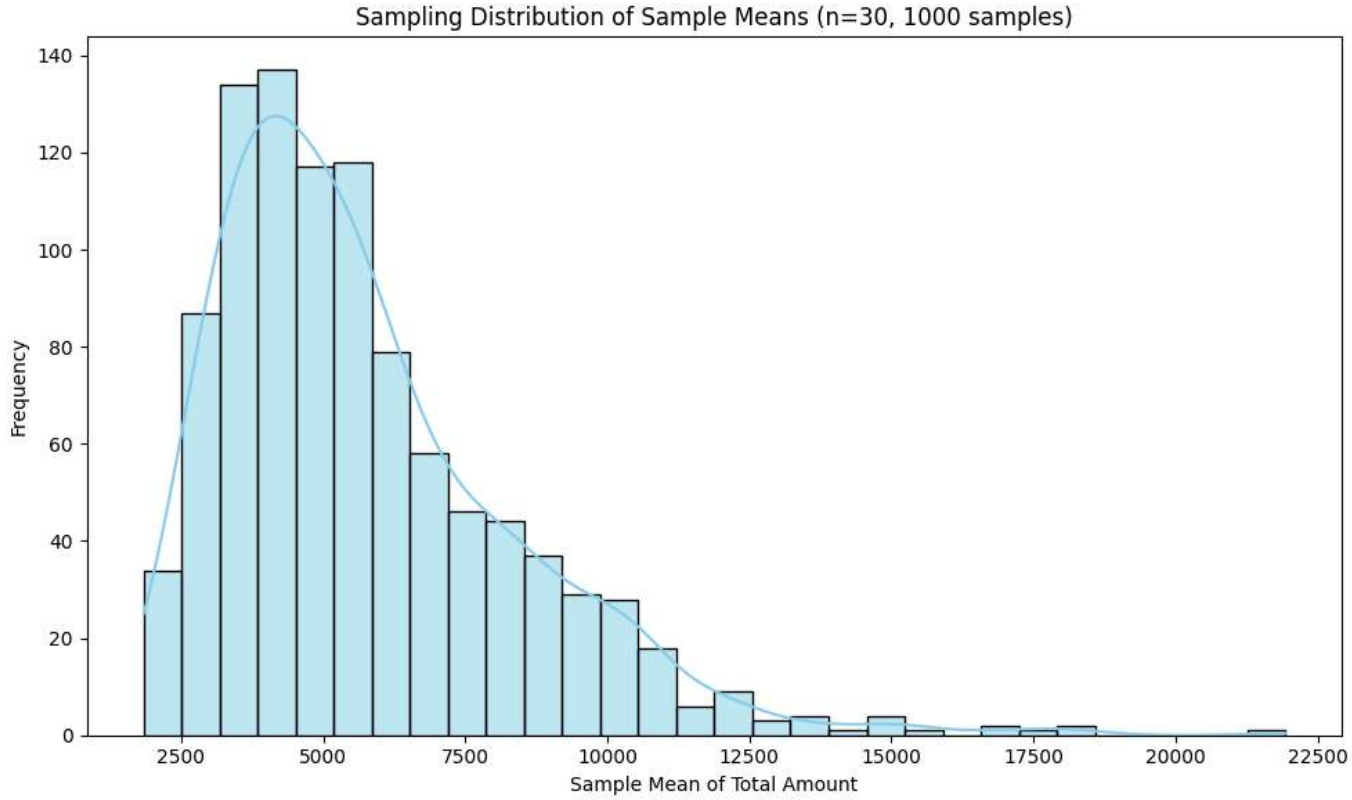
```
# Assume total_amount is Normally distributed (use sample mean & std). Find P(order > ₹4000). Visualise the curve with the shaded region.
import numpy as np
```

```
import matplotlib.pyplot as plt
from scipy.stats import norm
mean_val = orders["total_amount"].mean()
std_val  = orders["total_amount"].std()
print("Mean:", mean_val, "Std Dev:", std_val)
prob_gt_4000 = 1 - norm.cdf(4000, loc=mean_val, scale=std_val)
print("Probability order > ₹4000:", prob_gt_4000)
x = np.linspace(mean_val - 4*std_val, mean_val + 4*std_val, 500)
y = norm.pdf(x, mean_val, std_val)
plt.figure(figsize=(10,6))
plt.plot(x, y, color="blue")
x_shade = np.linspace(4000, mean_val + 4*std_val, 200)
y_shade = norm.pdf(x_shade, mean_val, std_val)
plt.fill_between(x_shade, y_shade, color="orange", alpha=0.5)
plt.axvline(4000, color="red", linestyle="--", label="Threshold ₹4000")
plt.title("Normal Distribution of Total Amount\nShaded Region = P(X > ₹4000)")
plt.xlabel("Total Amount")
plt.ylabel("Density")
plt.legend()
plt.tight_layout()
plt.show()
```

```
Mean: 5707.12928125 Std Dev: 13587.968113170553
Probability order > ₹4000: 0.5499897147160334
```



```
# Take 1000 random samples (n=30) from total_amount. Plot the sampling distribution of sample means. Does it look Normal? Why?
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns
data = orders["total_amount"].values
# Draw 1000 random samples of size n=30
sample_means = []
for _ in range(1000):
    sample = np.random.choice(data, size=30, replace=True)
    sample_means.append(sample.mean())
plt.figure(figsize=(10,6))
sns.histplot(sample_means, bins=30, kde=True, color="skyblue")
plt.title("Sampling Distribution of Sample Means (n=30, 1000 samples)")
plt.xlabel("Sample Mean of Total Amount")
plt.ylabel("Frequency")
plt.tight_layout()
plt.show()
```




```
# Compute a 95% confidence interval for the mean total_amount. Interpret: 'We are 95% confident that...'
import numpy as np
from scipy import stats
mean_val = orders["total_amount"].mean()
std_val  = orders["total_amount"].std(ddof=1)
n        = orders["total_amount"].shape[0]
```

```
se = std_val / np.sqrt(n)
ci_low, ci_high = stats.t.interval(
    confidence=0.95, df=n-1, loc=mean_val, scale=se
)
print("Mean:", mean_val)
print("95% CI:", (ci_low, ci_high))
```

```
Mean: 5707.12928125
95% CI: (np.float64(5409.330211752377), np.float64(6004.928350747622))
```

```
# Test: 'Does average order value differ between Gold and Platinum customers?' Set α = 0.05. State H0 and H1, run test, interpret p-value.
from scipy import stats
import pandas as pd
customer_spending = orders.groupby('customer_id')['total_amount'].sum().reset_index()
customer_spending.rename(columns={'total_amount': 'total_customer_spending'}, inplace=True)
customers = pd.merge(customers, customer_spending, on='customer_id', how='left')
customers['total_customer_spending'] = customers['total_customer_spending'].fillna(0)
platinum_threshold = customers['total_customer_spending'].quantile(0.90)
gold_threshold     = customers['total_customer_spending'].quantile(0.70)
def assign_customer_tier(spending):
    if spending >= platinum_threshold:
        return 'Platinum'
    elif spending >= gold_threshold:
        return 'Gold'
    else:
        return 'Standard'
customers['customer_tier'] = customers['total_customer_spending'].apply(assign_customer_tier)
orders_with_tiers = pd.merge(orders, customers[['customer_id', 'customer_tier']], on='customer_id', how='left')
gold_orders = orders_with_tiers.loc[orders_with_tiers["customer_tier"]=="Gold", "total_amount"]
platinum_orders = orders_with_tiers.loc[orders_with_tiers["customer_tier"]=="Platinum", "total_amount"]
if gold_orders.empty or platinum_orders.empty:
    print("Not enough 'Gold' or 'Platinum' orders to perform t-test. Please check tier assignment or data.")
else:
    t_stat, p_val = stats.ttest_ind(gold_orders, platinum_orders, equal_var=False)
    print("t-statistic:", t_stat)
    print("p-value:", p_val)
```

```
t-statistic: -10.301295592779766
p-value: 6.908487165105865e-24
```

```
# Test: 'Is average delivery_days the same across all 5 regions?' Use a one-way ANOVA. Interpret F-statistic and p-value.
from scipy import stats
groups = [orders.loc[orders["region"]==r, "delivery_days"].dropna()
          for r in orders["region"].unique()]
groups = [group for group in groups if not group.empty]
if len(groups) >= 2:
    F_stat, p_val = stats.f_oneway(*groups)
    print("F-statistic:", F_stat)
    print("p-value:", p_val)
else:
    print("Not enough valid groups (regions with non-NaN delivery_days) to perform ANOVA.")
```

```
F-statistic: 0.6128506511746126
p-value: 0.6533728016003226
```

```
# One-sample t-test: The company claims average delivery is ≤ 5 days. Test this at α = 0.05.
from scipy import stats
sample = orders["delivery_days"]
t_stat, p_val_two_tailed = stats.ttest_1samp(sample, popmean=5)
if t_stat > 0:
    p_val = p_val_two_tailed / 2
else:
    p_val = 1 - (p_val_two_tailed / 2)
print("t-statistic:", t_stat)
print("one-tailed p-value:", p_val)
```

```
t-statistic: nan
one-tailed p-value: nan
```

```
# From marketing_df, compare ROAS between 'Control' and 'Treatment' groups. Run a two-sample t-test. Is the treatment statistically better? Calculate effect size (Cohe
from scipy import stats
import numpy as np
control_roas   = marketing.loc[marketing["ab_group"]=="Control", "revenue_generated"]
treatment_roas = marketing.loc[marketing["ab_group"]=="Treatment", "revenue_generated"]
if control_roas.empty or treatment_roas.empty:
    print("Not enough 'Control' or 'Treatment' ROAS data to perform t-test. Please check your data or group filtering criteria.")
else:
    t_stat, p_val = stats.ttest_ind(treatment_roas, control_roas, equal_var=False)
    print("t-statistic:", t_stat)
    print("p-value:", p_val)
    mean_control   = control_roas.mean()
    mean_treatment = treatment_roas.mean()
    diff           = mean_treatment - mean_control
    n1, n2 = len(control_roas), len(treatment_roas)
    s1, s2 = control_roas.std(ddof=1), treatment_roas.std(ddof=1)
    if n1 > 1 and n2 > 1:
        pooled_std = np.sqrt(((n1-1)*s1**2 + (n2-1)*s2**2) / (n1+n2-2))
        cohens_d = diff / pooled_std
        print("Mean Control:", mean_control)
        print("Mean Treatment:", mean_treatment)
        print("Difference:", diff)
        print("Cohen's d:", cohens_d)
    else:
        print("Cannot calculate Cohen's d: Not enough data points in one or both groups.")
```

```
t-statistic: 0.946303137749667
p-value: 0.3451851479350898
Mean Control: 3758677.6222222224
Mean Treatment: 4491360.672727273
Difference: 732683.0505050505
Cohen's d: 0.1342167371132562
```

```
# Compute sample size needed to detect a 10% lift in ROAS with 80% power at α=0.05. Use the formula or statsmodels TTestIndPower.
from statsmodels.stats.power import TTestIndPower
alpha = 0.05
power = 0.80
effect_size = 0.10
analysis = TTestIndPower()
sample_size = analysis.solve_power(effect_size=effect_size,
                                   alpha=alpha,
                                   power=power,
```

```
                                    alternative='two-sided')
print("Required sample size per group:", sample_size)
```

```
Required sample size per group: 1570.7330430520028
```